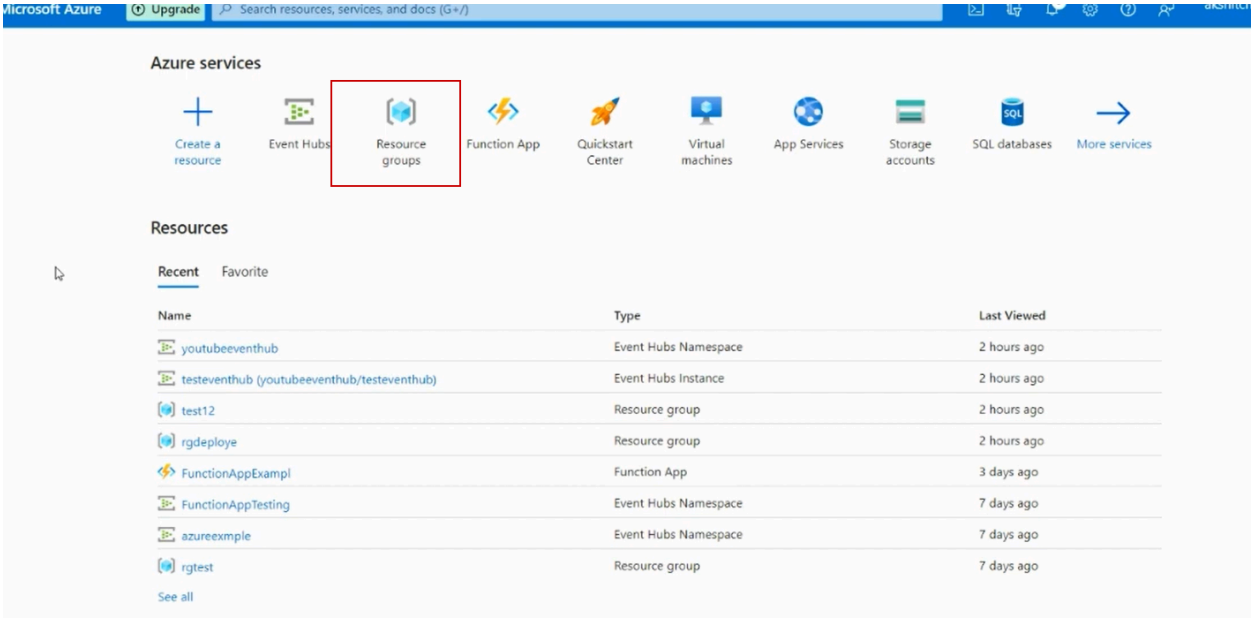
Event Hub

Status	Done

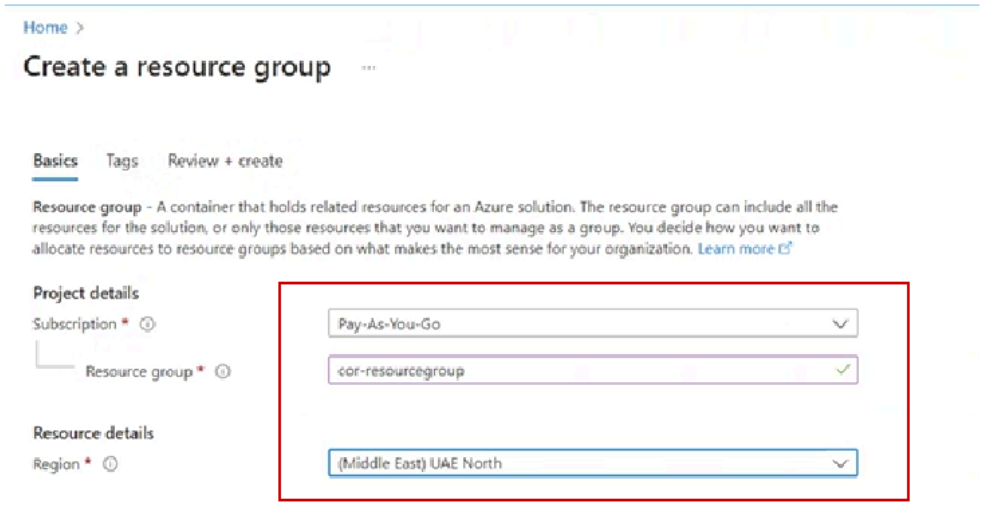
1- open Portal

2- Create a Resource Group



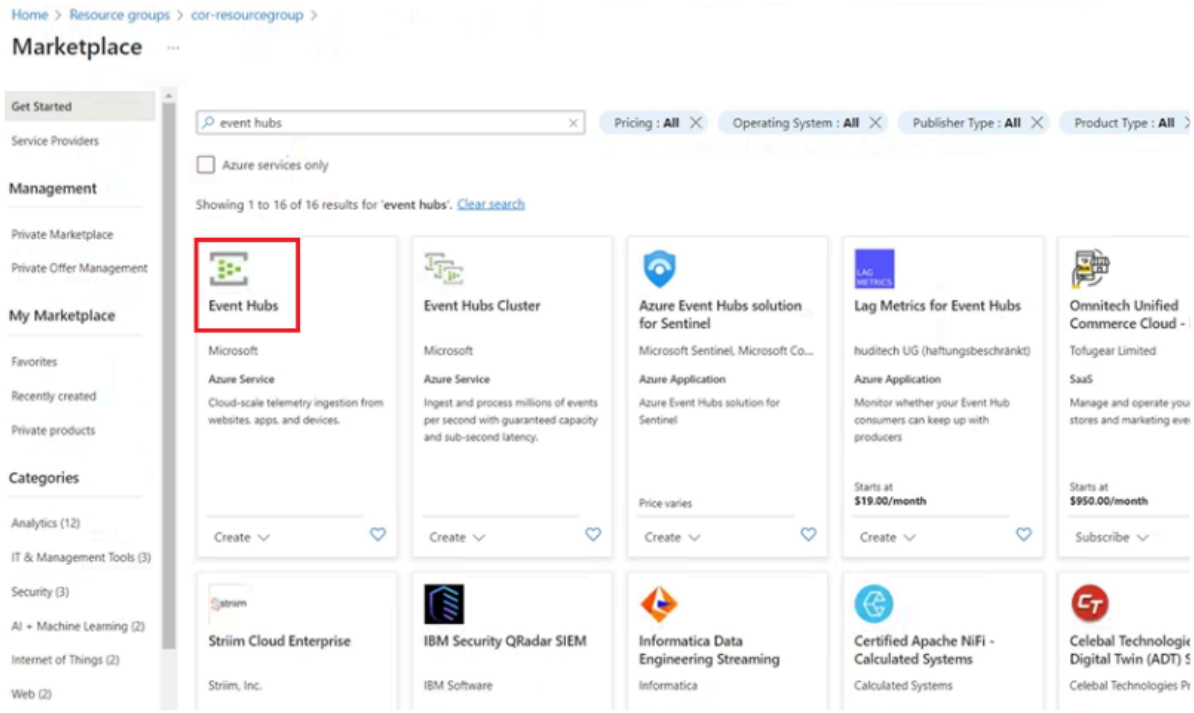
3- Add the following details

- Subscription : `Pay-As-You-Go`
- Resource Group : `cor-resourcegroup`
- Region : `UAE North`



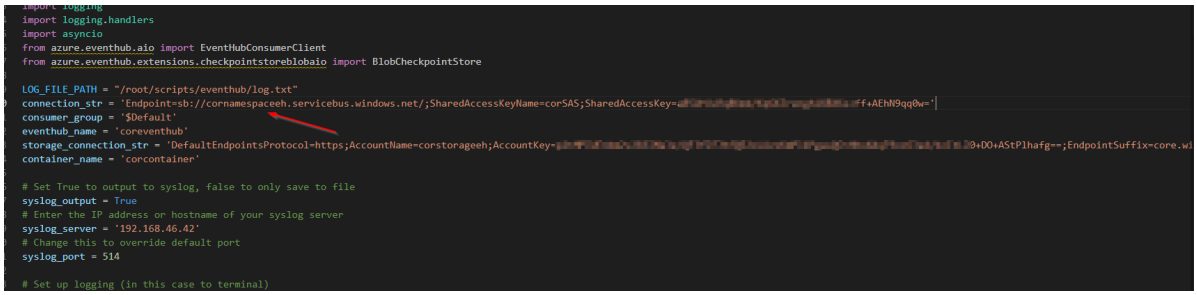
- Click : view + Create
- then create

4- Search with Event Hub

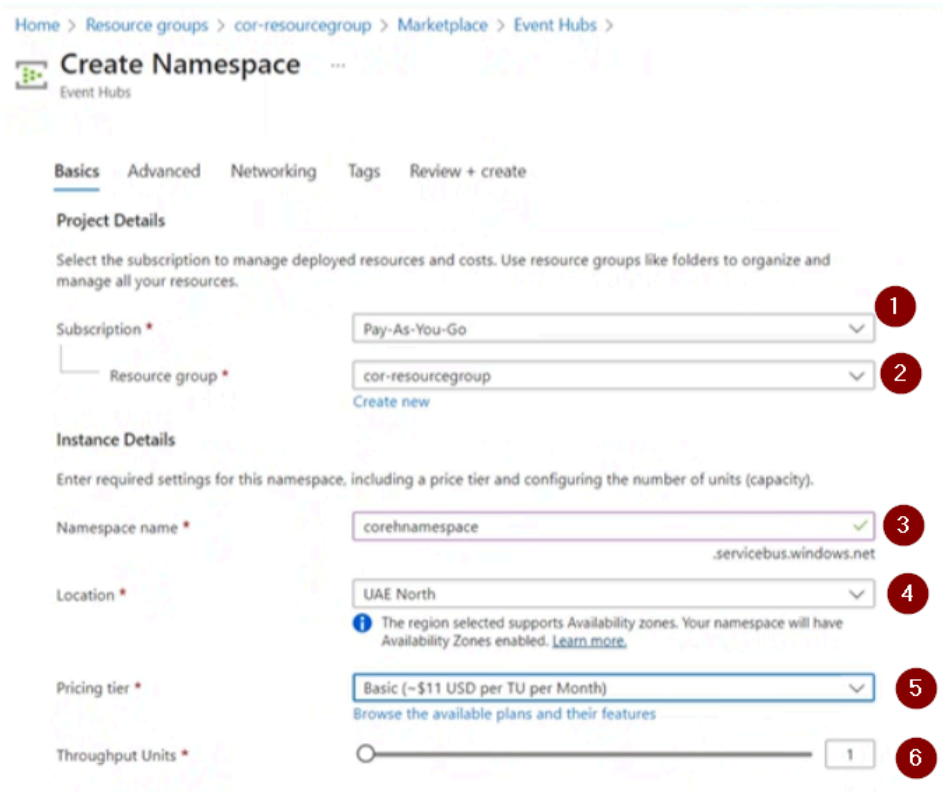


5- Create the Name space with the following Details :

- Subscription : `Pay-As-You-Go`
- Resource Group : `cor-resourcegroup`
- Namespace Name : `corehnamespace` → Send in Chat (1)



- Region : `UAE North`
- Pricing Tier : `Basic`

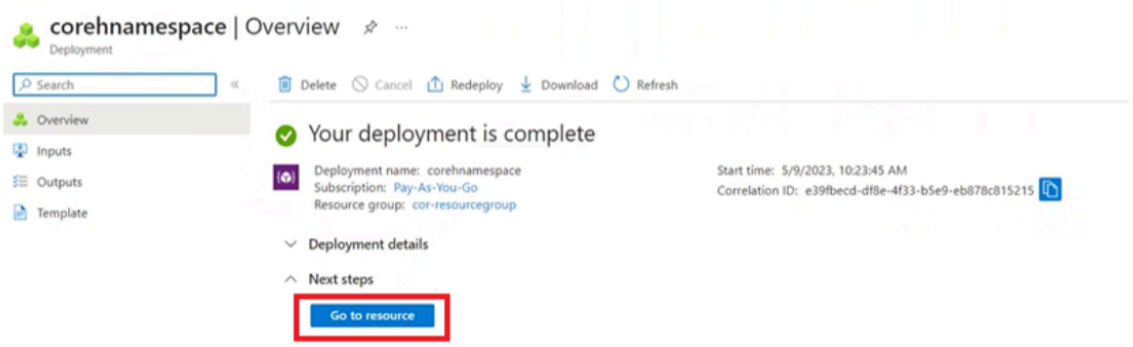


- Click : view + Create
- then create

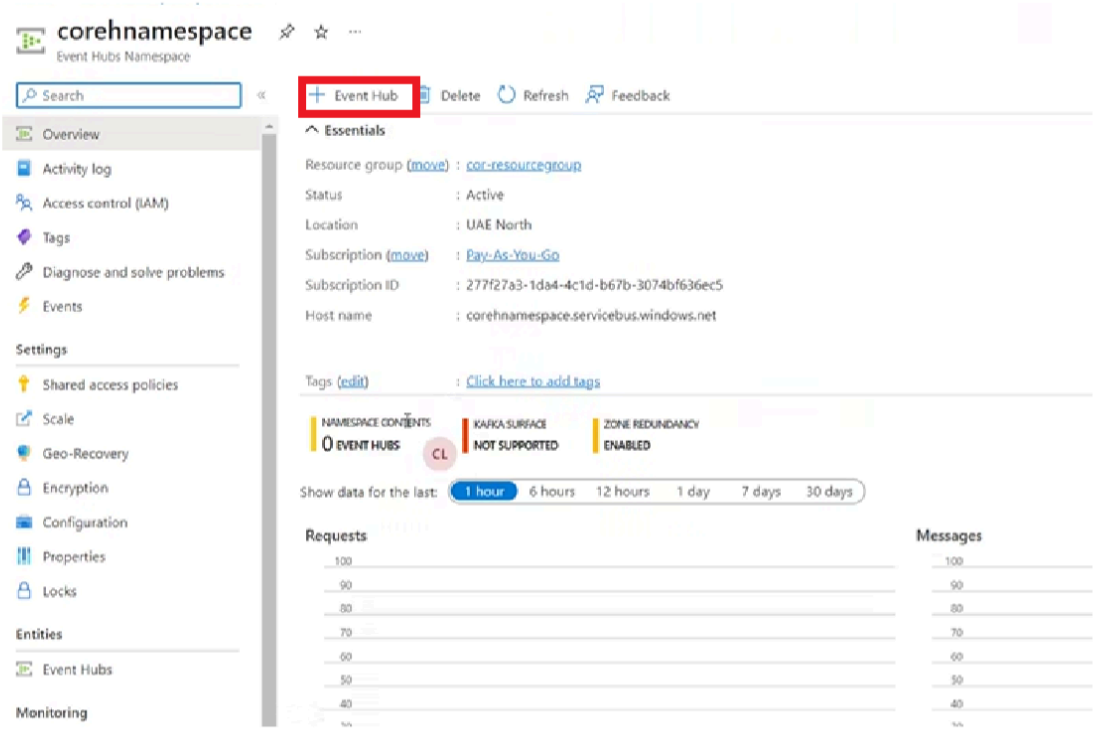
> ⌛ will take Couple of minutes , wait for deployment to finish

After completion → Click `“Go To Resources”`

Go to Resource

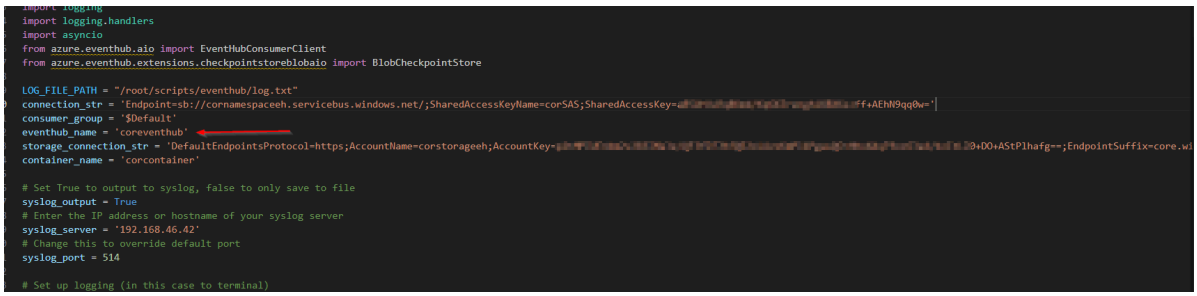


6- Create EventHub

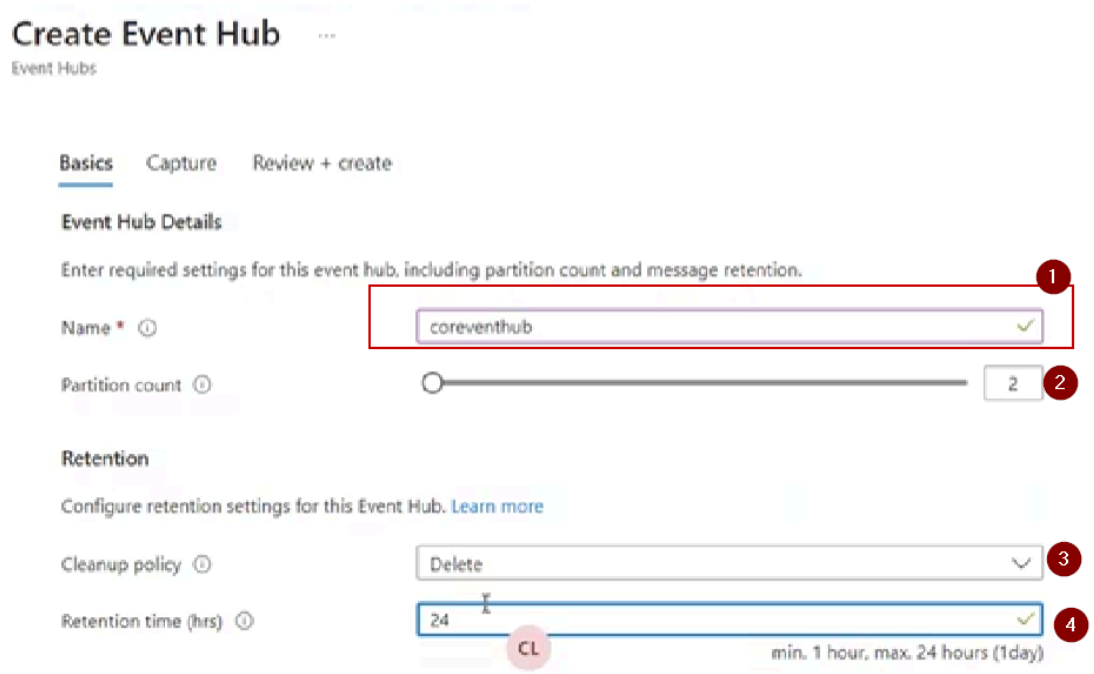


Add the following Details:

- Name : `coreventhub` → Send in Chat (2)



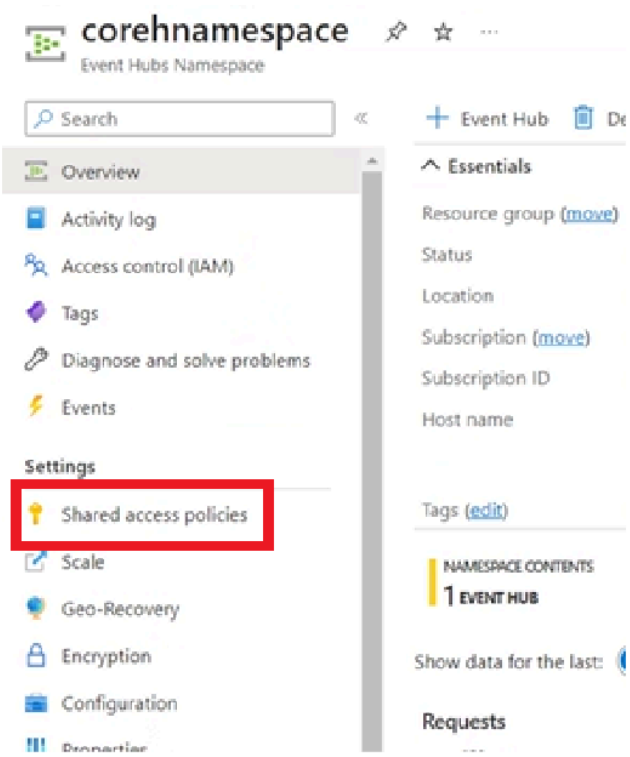
- Partition Count : `2`
- Clean up policy : `Delete`
- Retention time : `24`



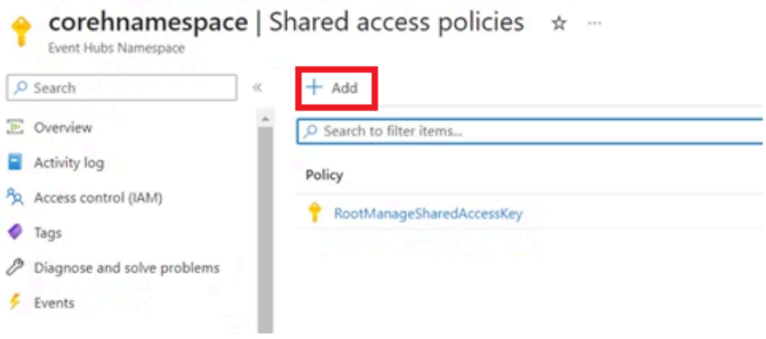
- Click : view + Create
- then create

7- From the Side Menu

Click Shared Access Policy



then add



Add the Following Details :

- Name : `corehSAS` → (called in Script : Shared Access Key Name) send in chat (3)

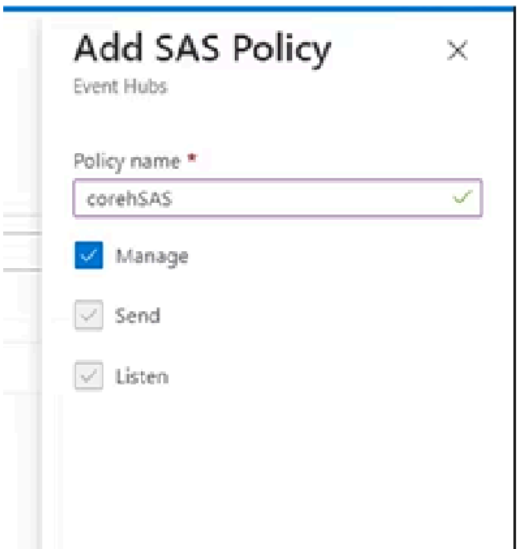
```
import logging.handlers
import asyncio
from azure.eventhub.aio import EventHubConsumerClient
from azure.eventhub.extensions.checkpointstoreblobaio import BlobCheckpointStore

LOG_FILE_PATH = "/root/scripts/eventhub/log.txt"
connection_str = 'Endpoint=sb://cornamespaceeh.servicebus.windows.net/;SharedAccessKeyName=corSAS;SharedAccessKey=[redacted]'
consumer_group = '$Default'
eventhub_name = 'coreventhub'
storage_connection_str = 'DefaultEndpointsProtocol=https;AccountName=corstorageeh;AccountKey=[redacted]'
container_name = 'corcontainer'

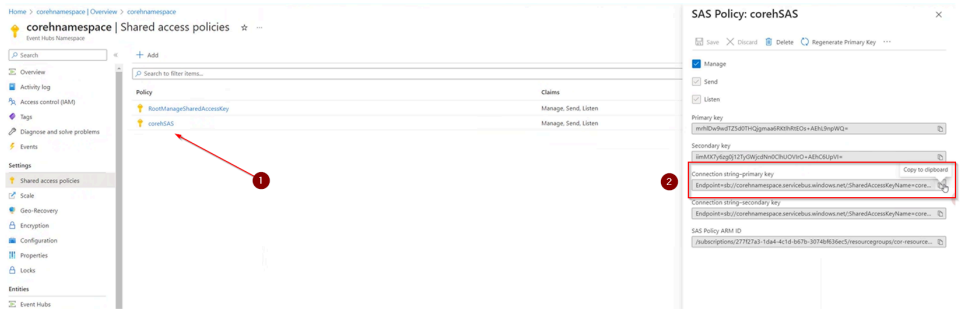
# Set True to output to syslog, false to only save to file
syslog_output = True
# Enter the IP address or hostname of your syslog server
syslog_server = '192.168.46.42'
# Change this to override default port
syslog_port = 514

# Set up logging (in this case to terminal)
```

- Select : `Manage`



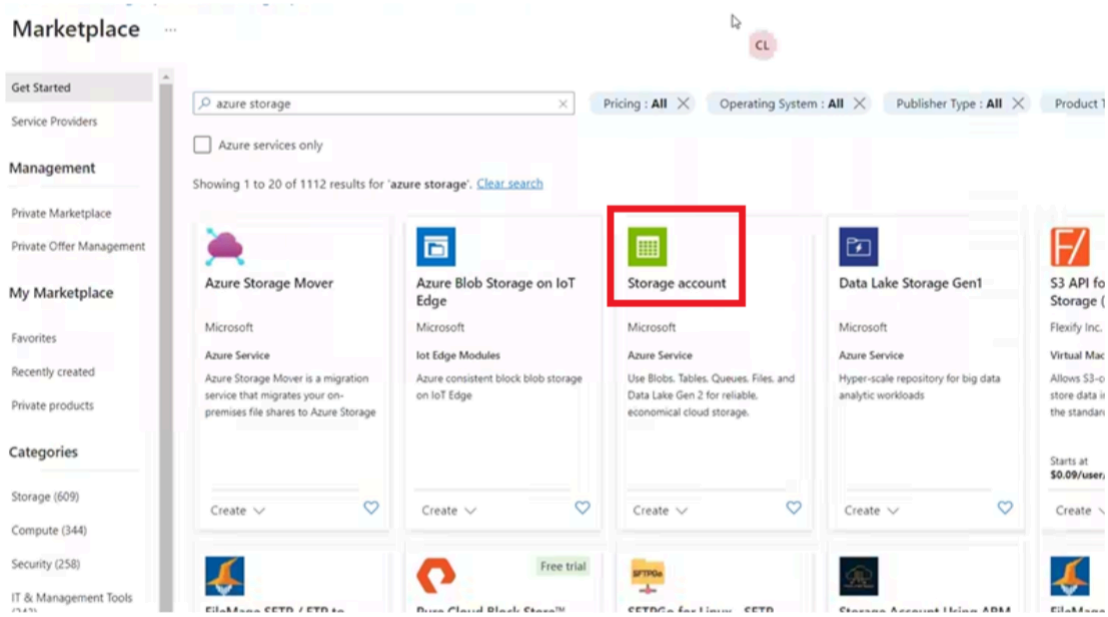
- Click Create
- then click on it after Creation
- Copy `Connection-String Primary Key` (Called in Script Shared Access Key) → send in chat (4)





8- Then Search For Azure Storage

Search for Azure Storage



then add the following Details :

- Subscription : `Pay-As-You-Go`
- Resource Group : `cor-resourcegroup`
- Storage Account name : `corstorage` → Send in Chat (5)

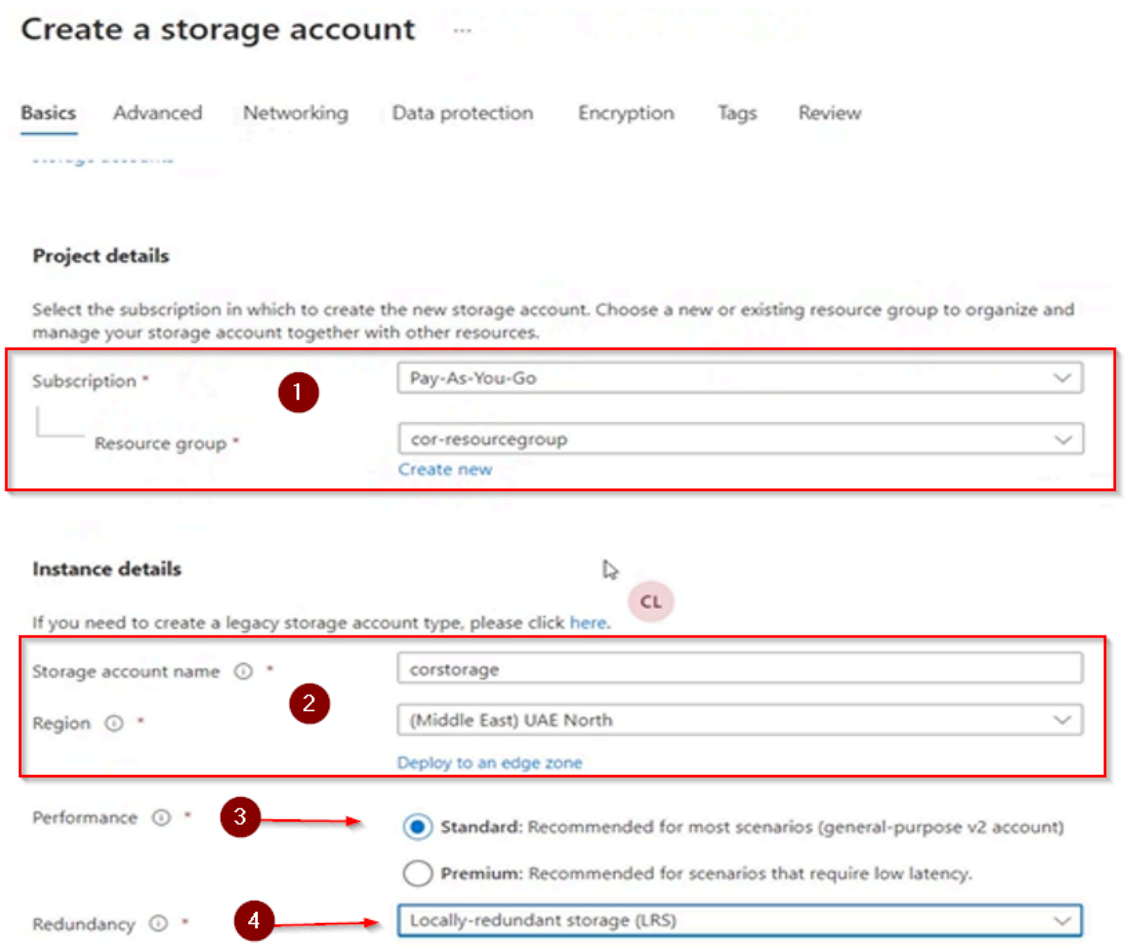
```
import logging.handlers
import asyncio
from azure.eventhub.aio import EventHubConsumerClient
from azure.eventhub.extensions.checkpointstoreblobaio import BlobCheckpointStore

LOG_FILE_PATH = "/root/scripts/eventhub/log.txt"
connection_str = 'Endpoint=sb://cornamespaceeh.servicebus.windows.net/;SharedAccessKeyName=corSAS;SharedAccessKey=[redacted]ff+AEhN9qq0w='
consumer_group = '$Default'
eventhub_name = 'coreventhub'
storage_connection_str = 'DefaultEndpointsProtocol=https;AccountName=corstorageeh;AccountKey=[redacted]9+DO+AStPlhafg==;EndpointSuffix=core.wi
container_name = 'corcontainer'

# Set True to output to syslog, false to only save to file
syslog_output = True
# Enter the IP address or hostname of your syslog server
syslog_server = '192.168.46.42'
# Change this to override default port
syslog_port = 514

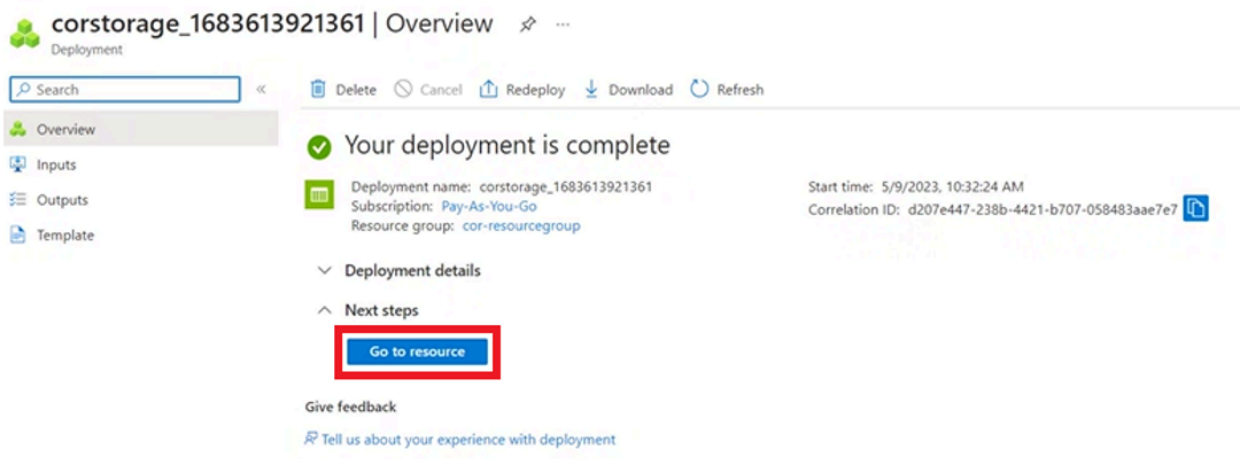
# Set up logging (in this case to terminal)
```

- Region : `UAE North`
- Performance : `Standard`
- Redundancy : `Locally-Redundant storage (LRS)`

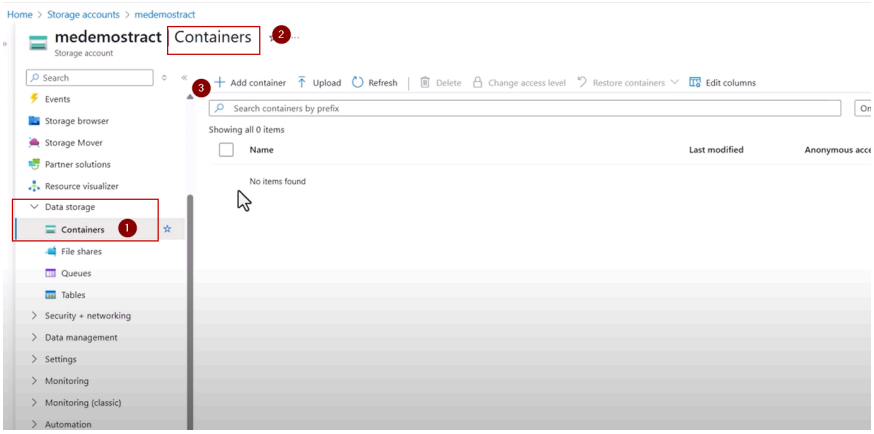


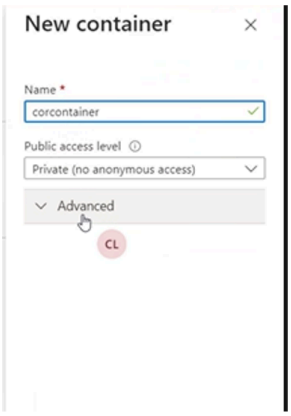
- Click : view + Create
- then create

Wait Until Deployment Completes then → `Go to Resource`



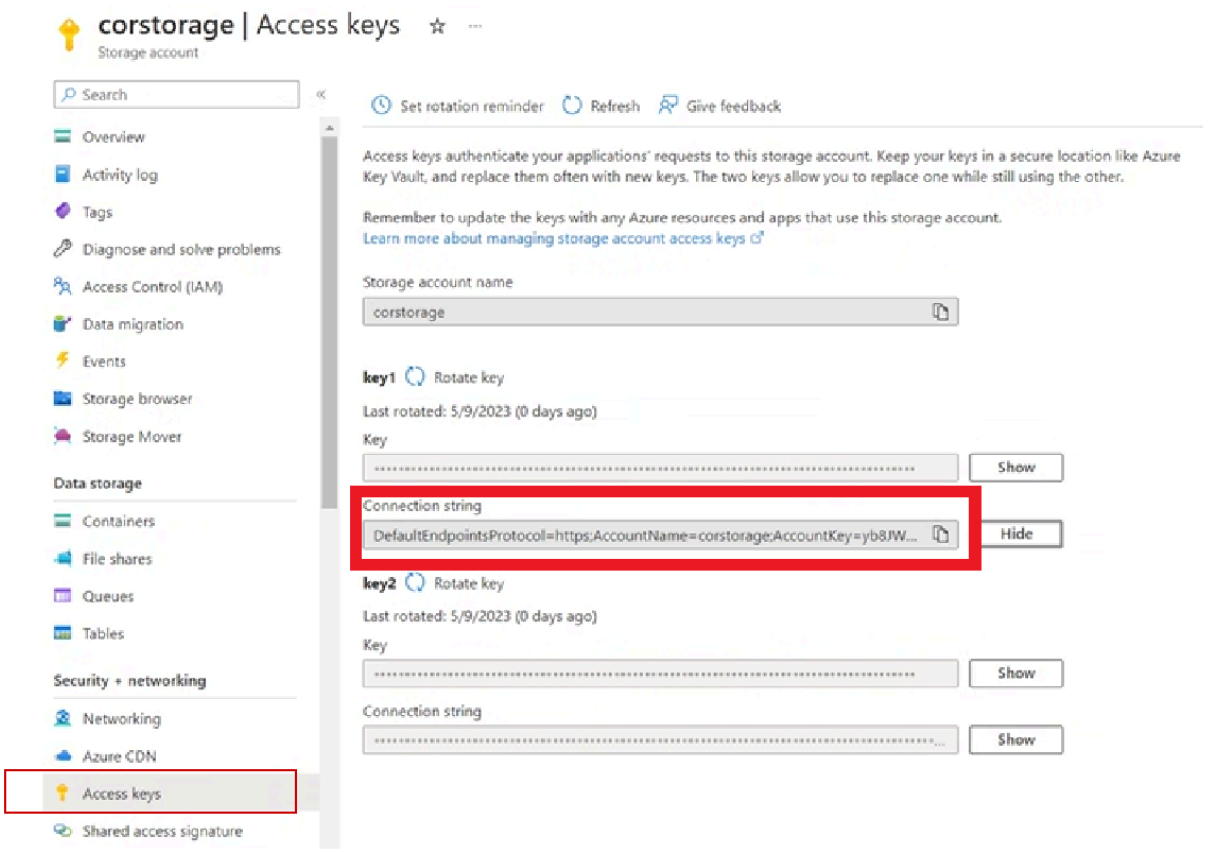
- from Side bar : Click Containers
- Click Add Container
- Name : `corcontainer` → send in chat (6)

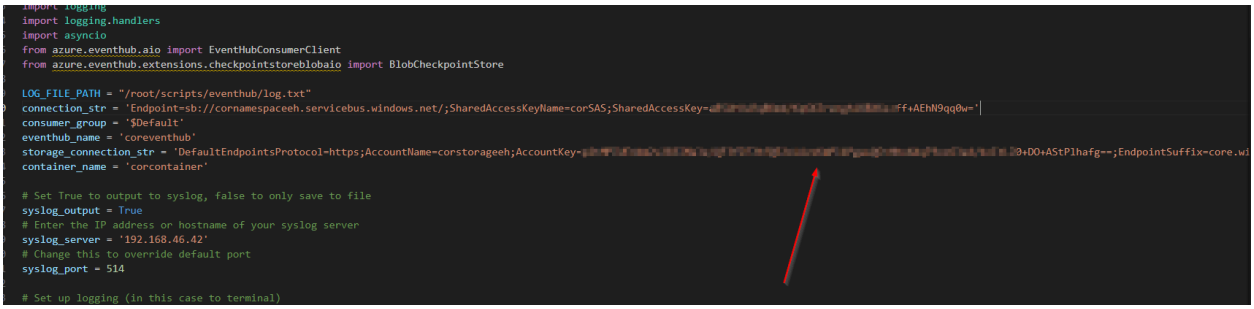




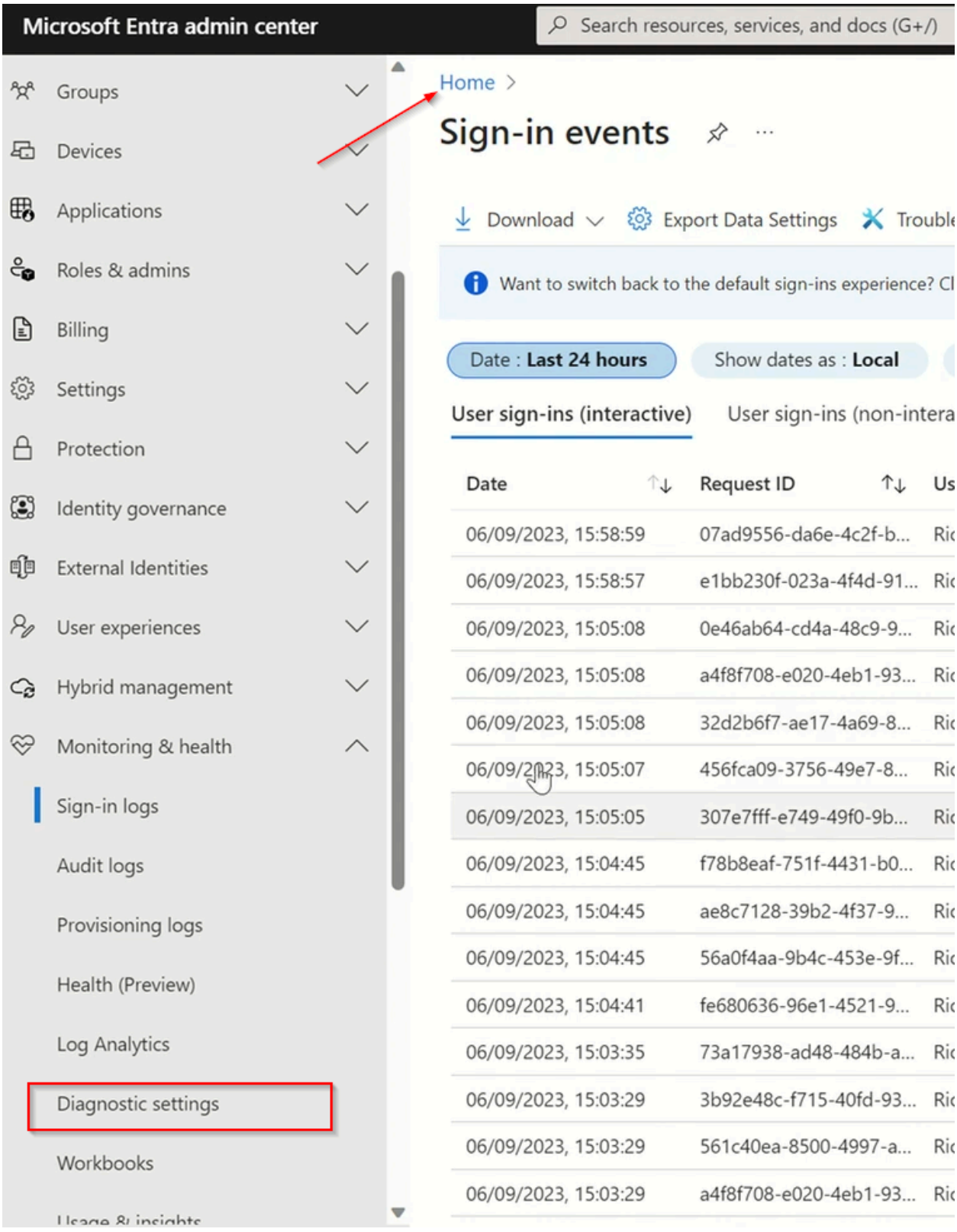


- From Side tab : Click on `Access Key`
- then copy `Connection String` → (Called in Script : Account Key) send in chat (7)

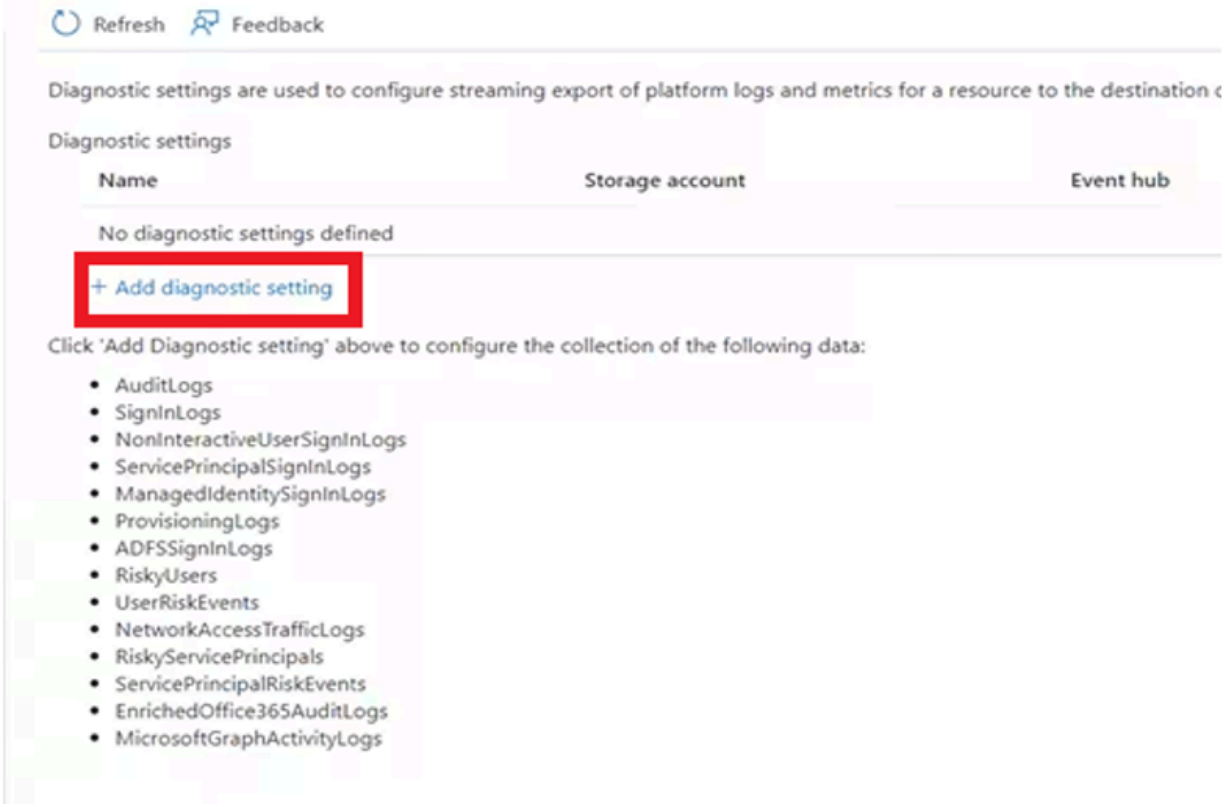




9- Back to Entra ID Home → Open Diagnostic Settings



- Click Add diagnostic settings



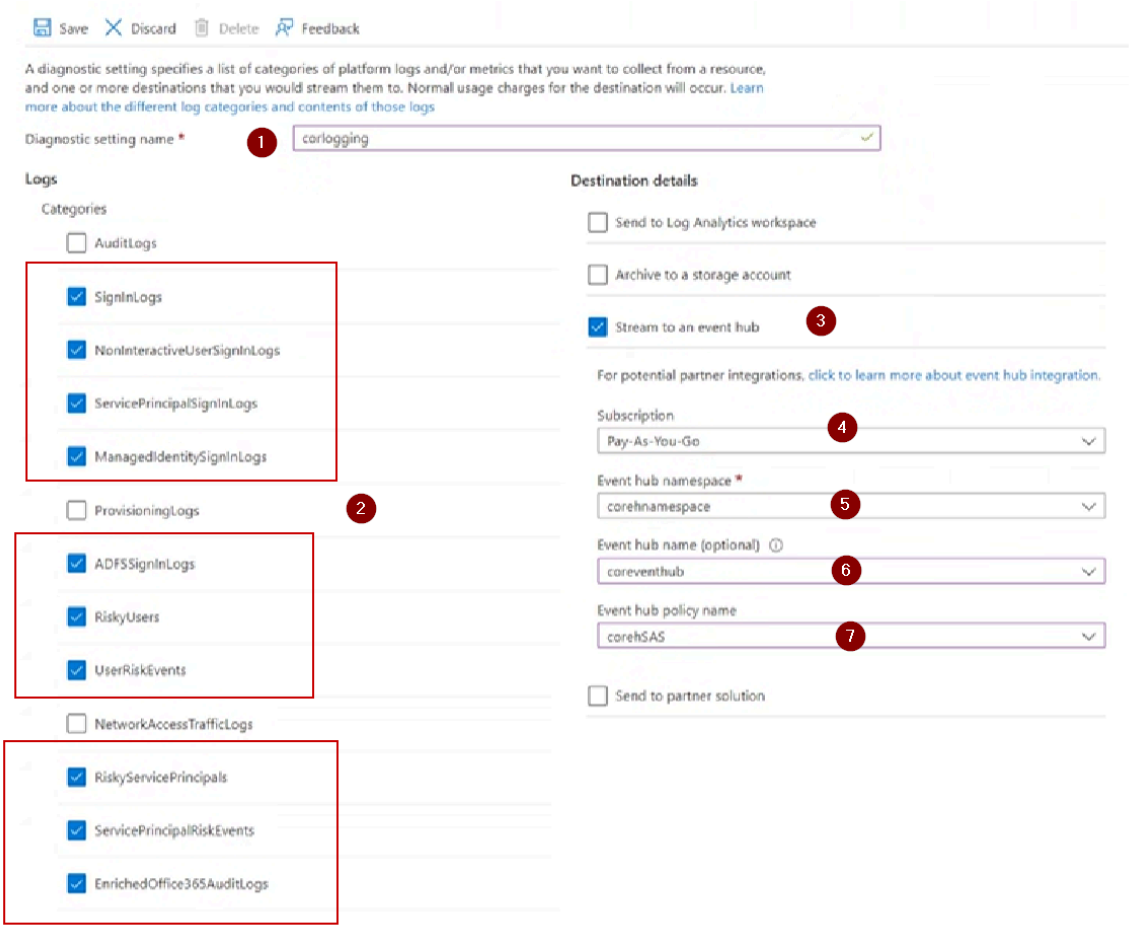
Add the Following Details :

1. Name: `corlogging`
2. Select the Following Logs :

```
Audit Logs
Sign In Logs
Non Interactive User Sign In Logs
Service Principal Sign In Logs
Managed Identity Sign In Logs
Provisioning Logs
ADFS Sign In Logs
Risky Users
User Risk Events
Network Access Traffic Logs
Risky Service Principals
Service Principal Risk Events
Enriched Office365 Audit Logs
```

```
Microsoft Graph Activity Logs
Remote Network Health Logs
Network Access Alerts
Network Access Connection Events
AzureAD Graph Activity Logs
Microsoft Service principal sign in logs
```

1. Check : `Stream on EventHub`
2. Subscription : `Pay-As-You-Go`
3. Eventhub Namespace : `corehnamespace`
4. Eventhub Name : `coreventhub`
5. Eventhub Policy Name : `corehSAS`



```
import os
import json
import logging
import logging.handlers
import asyncio
from azure.eventhub.aio import EventHubConsumerClient
from azure.eventhub.extensions.checkpointstoreblobaio import
BlobCheckpointStore

LOG_FILE_PATH = "/root/scripts/eventhub/log.txt"
connection_str = 'Endpoint=sb://cor-eventhub-namespace-01.ser
vicebus.windows.net/;SharedAccessKeyName=cor-sas-01;SharedAcc
essKey=AEhE1snvU='
consumer_group = '$Default'
eventhub_name = 'cor-eventhub-01'
storage_connection_str = 'DefaultEndpointsProtocol=https;Acco
untName=corstorage01;AccountKey=NveoImnwoEc/KBh3GQ==;Endpoint
Suffix=core.windows.net'
container_name = 'cor-container-01'

# Set True to output to syslog, false to only save to file
syslog_output = True
# Enter the IP address or hostname of your syslog server
syslog_server = '10.100.104.12'
# Change this to override default port
syslog_port = 514

# Set up logging (in this case to terminal)
log = logging.getLogger(__name__)
log.root.setLevel(logging.DEBUG)
```

```
log_formatter = logging.Formatter('%(levelname)s %(message)
s')
log_handler = logging.StreamHandler()
log_handler.setFormatter(log_formatter)
log.addHandler(log_handler)

# Set up syslog output
syslog_handler = logging.handlers.SysLogHandler(address=(sysl
og_server, syslog_port))
syslog_formatter = logging.Formatter('%(message)s')
syslog_handler.setFormatter(syslog_formatter)
syslogger = logging.getLogger(__name__)
syslogger = logging.getLogger('SysLogger')
syslogger.addHandler(syslog_handler)

# Set up file logging
filelog_handler = logging.FileHandler(LOG_FILE_PATH)
filelog_formatter = logging.Formatter('%(message)s')
filelog_handler.setFormatter(filelog_formatter)
filelog = logging.getLogger('FileLogger')
filelog.root.setLevel(logging.DEBUG)
filelog.addHandler(filelog_handler)

async def on_event_batch(partition_context, events):
    try:
        log.debug("Received event from partition {}".format(p
artition_context.partition_id))
        #filelog.debug("Received event from partition {}".for
mat(partition_context.partition_id))
        for event in events:
            #filelog.debug('Before dumping json')
            body = json.dumps(json.loads(event.body_as_str())
["records"][0])

            #filelog.debug('Before log handling')
            if syslog_output is True:
```

```
                #filelog.debug('Before syslog forwarding')
                syslogger.info(body)
            else:
                log.debug("New incoming event:")
                log.debug(event)

            #filelog.debug('Before update_checkpoint()')
            await partition_context.update_checkpoint(event)
            #filelog.debug('After update_checkpoint()')
    except Exception as e:
        filelog.debug('Error in on_event_batch: ' + str(e))

async def receive_batch():
    try:
        #filelog.debug('Before checkpoint_store initializatio
n')
        checkpoint_store = BlobCheckpointStore.from_connectio
n_string(storage_connection_str, container_name)
        #filelog.debug('Before client initialization')
        client = EventHubConsumerClient.from_connection_strin
g(connection_str, consumer_group, eventhub_name=eventhub_nam
e, checkpoint_store=checkpoint_store)
        async with client:
            #filelog.debug('Before receive_batch')
            await asyncio.wait_for (
                client.receive_batch(
                    on_event_batch=on_event_batch,
                    max_batch_size=1000,
                    starting_position="-1",  # "-1" is from t
he beginning of the partition.
                ),
                timeout=295
            )
            #filelog.debug('After receive_batch')
            await client.close()
            # receive events from specified partition:
```

```
            # await client.receive_batch(on_event_batch=on_ev
ent_batch, partition_id='0')
    except Exception as e:
        filelog.debug('Error in receive_batch:' + str(e))

if __name__ == '__main__':
    try:
        #filelog.debug('Before get_event_loop()')
        loop = asyncio.get_event_loop()
        #filelog.debug('Before loop.run_until_complete()')
        loop.run_until_complete(receive_batch())
        #filelog.debug('After loop.run_until_complete()')
        loop.close()
        quit()
    except Exception as e:
        filelog.debug('Error in Main: ' + str(e))
```

```
Event Hub Namespacefor SUB_ID in $(az account list --query "
[].id" -o tsv); do echo "Processing: $SUB_ID" && az monitor d
iagnostic-settings subscription create --name "$DIAG_SETTING_
NAME" --subscription "$SUB_ID" --event-hub-name "$EVENT_HUB_N
AME" --event-hub-auth-rule "$EVENT_HUB_RULE_ID" --location "u
aenorth" --logs '[{"category":"Administrative","enabled":tru
e},{"category":"Security","enabled":true},{"category":"Aler
t","enabled":true},{"category":"Recommendation","enabled":tru
e},{"category":"Policy","enabled":true}]' && echo "Done: $SUB
_ID" && echo "---"; done Connection String:
```